SEMANTIC RESEARCH PAPER RETRIEVAL ENGINE

A Hybrid Information Retrieval System Using BM25 and Sentence Embeddings

Submitted by

Pranav GS – 2023510053

Srimathi S – 2023510055

Deeksha Lakshana K - 2023510048

B.Tech. Artificial Intelligence and Data Science

Department of Information Technology

Madras Institute of Technology

Anna University

Abstract

Research literature is expanding rapidly, making it difficult for students and researchers to locate papers that are relevant to a question rather than merely containing the same keywords. This project implements a lightweight research-paper retrieval engine that collects paper metadata from arXiv and ranks papers using a hybrid combination of lexical retrieval and semantic similarity. The data-collection component searches ten configured topics, stores selected metadata, removes records with missing essential text, and deduplicates by paper identifier and title. The retrieval component builds a BM25 index over each paper title and abstract and creates sentence embeddings using the sentence-transformers/all-MiniLM-L6-v2 model. For a user query, the system computes lexical and semantic scores, independently min–max normalizes the scores, and fuses them through a configurable weight α. The user can also restrict the publication-year range and select a Top-K value of five or ten. A Gradio interface presents ranked titles, authors, publication year, venue, abstract excerpts, component scores, and a link to the paper PDF.

The implementation demonstrates an end-to-end information-retrieval workflow: metadata acquisition, text preparation, indexing, query-time scoring, rank fusion, filtering, and web-based presentation. It also includes a small parameter-sweep experiment that plots Mean Reciprocal Rank (MRR) for four values of α. The supplied source code does not include execution logs or the generated metric values; therefore, this report explains the evaluation procedure without asserting unobserved performance results. A key methodological limitation is that the current "ground truth" is derived from BM25's own top-ranked papers, so the MRR experiment measures agreement with a lexical pseudo-label rather than independently judged relevance.

Keywords

Information retrieval; hybrid search; BM25; sentence embeddings; Sentence Transformers; arXiv API; relevance ranking; Gradio; MRR.

1. Introduction

1.1 Background

Academic research relies on searching, screening, and comparing scholarly publications. Traditional keyword search is useful when a query contains the exact vocabulary used in a title or abstract, but it can miss conceptually related work when authors use synonyms or different phrasing. For example, a query about “identifying malicious software” may be related to papers describing “malware detection,” even when the exact query wording is absent. Semantic retrieval represents text in a vector space so that related phrases can receive similar representations, while lexical retrieval remains valuable for exact technical terms and named methods [2, 3].

1.2 Problem statement

The project addresses the problem of retrieving a small, ranked set of relevant papers from a topic-focused corpus using one text query and optional publication-year constraints. The system must combine complementary retrieval signals, preserve paper metadata for inspection, and provide an interface that does not require the user to interact directly with Python code.

1.3 Aim and objectives

- Collect research-paper metadata from arXiv across a configurable set of artificial-intelligence and related topics.
- Clean essential fields and remove exact duplicate identifiers and titles before indexing.
- Build a BM25 lexical index from title and abstract tokens.
- Generate normalized sentence embeddings for paper text using all-MiniLM-L6-v2.
- Fuse lexical and semantic scores with a tunable parameter α and apply year filtering.
- Present ranked results through a Gradio user interface with metadata and PDF links.
- Explore the effect of α with an MRR-based parameter sweep and identify evaluation limitations.

1.4 Scope

The current implementation is a research prototype, not a production-scale scholarly search service. Its corpus is drawn from arXiv search results for ten predefined topics: deep learning, computer vision, natural language processing, generative AI, cybersecurity, anomaly detection, transformer models, reinforcement learning, medical information retrieval, and machine learning. Each topic requests up to 50 results sorted by relevance. Because topics overlap, requests can fail, and exact duplicates are removed, the final number of unique papers is data-dependent. The implementation does not crawl publisher websites, download and parse full-text PDFs, retrieve citation counts, or learn ranking weights from user feedback.

1.5 Intended users and applications

Potential users include students, research assistants, engineers conducting literature reviews, and practitioners seeking an initial reading list. Example uses are discovering papers about an unfamiliar concept, narrowing literature to a time window, and comparing lexical versus semantic retrieval behavior. Search results should be treated as a starting point for review, not as a substitute for systematic literature-search protocols.

2. Background and Related Work

2.1 Lexical retrieval and BM25

BM25 is a probabilistic term-matching ranking function that considers how often query terms occur in a document, how informative those terms are across the corpus, and document-length normalization. Its practical value is that it produces an interpretable lexical relevance score without requiring a trained neural model for every query [1]. The project uses the BM25Okapi implementation provided by rank_bm25. The library expects a tokenized document corpus; its documentation notes that preprocessing such as lowercasing, stop-word handling, and stemming is the caller's responsibility [4]. This project lowercases text and extracts word-like tokens with a regular expression, but it does not remove stop words, stem terms, or lemmatize words.

2.2 Sentence embeddings and semantic retrieval

Sentence-BERT-style models encode text into dense vectors intended to make semantic similarity search efficient [2]. The project uses sentence-transformers/all-MiniLM-L6-v2, a compact embedding model commonly used for semantic similarity and information-retrieval tasks [3]. Paper text is defined as the concatenation of the title and abstract. At query time, the query is encoded into a vector and compared with the precomputed document vectors using a dot product after normalization. For normalized vectors, the dot product corresponds to cosine similarity.

2.3 Hybrid retrieval

Lexical and semantic methods have complementary strengths. BM25 can strongly reward rare technical terms and exact terminology, whereas embeddings can capture meaning across related expressions. Hybrid retrieval combines these signals. In this implementation, both score arrays are min–max normalized to the interval [0, 1] and then combined by a weighted sum. The weight α controls the relative contribution of the semantic score. At $\alpha = 0$, the final score is entirely lexical; at $\alpha = 1$, it is entirely semantic.

2.4 Data source and presentation layer

The arXiv API supports search queries, result limits, and relevance-based sorting [5]. The code relies on the Python arxiv client to retrieve titles, abstracts, authors, publication timestamps, and PDF URLs. Gradio provides the interface layer: the Interface class maps Python function inputs and outputs to browser components, while launch(share=True) can create a public sharing link through a tunnel [6]. This enables rapid demonstration, but public sharing should be avoided for sensitive or non-public data unless suitable access controls are added.

2.5 Positioning of this project

The contribution is primarily an integrated prototype and a clear baseline for experimentation, rather than a new retrieval algorithm. The pipeline demonstrates how to join an open research metadata source, conventional lexical indexing, dense semantic vectors, score fusion, temporal filtering, a basic ranking experiment, and a browser UI in a single Python workflow. The central research question for a future evaluation is whether hybrid ranking improves independently assessed relevance over BM25-only and semantic-only baselines.

3. Requirements and System Architecture

3.1 Functional requirements

ID	Requirement	Implementation evidence
FR-01	Collect papers for configured topics	Ten-topic list and arxiv.Search loop; up to 50 results per topic.
FR-02	Clean and deduplicate metadata	Drop missing essentials, remove empty abstracts, deduplicate by paperId and title.
FR-03	Build lexical index	Tokenized title + abstract corpus passed to BM25Okapi.
FR-04	Build semantic index	MiniLM embeddings generated once for the corpus and normalized.
FR-05	Support user search and filters	Query box, minimum year, maximum year, Top-K, and α controls.
FR-06	Rank using two signals	Weighted sum of normalized semantic and BM25 scores.
FR-07	Display evidence for inspection	Title, authors, year, venue, score components, abstract excerpt, and PDF link.
FR-08	Run a small ranking experiment	MRR sweep over $\alpha \in \{0.3, 0.5, 0.7, 0.8\}$ and line plot.

3.2 Non-functional requirements

The prototype should be understandable, reproducible, reasonably responsive for a small corpus, and tolerant of individual topic-query failures. It should disclose the data source and distinguish model scores from true probabilities. For reliability, the actual dataset count, model identity, dependency versions, and evaluation outputs should be logged. For safety, externally retrieved strings should be escaped before HTML rendering and the public interface should not be used to expose confidential material.

3.3 Architecture

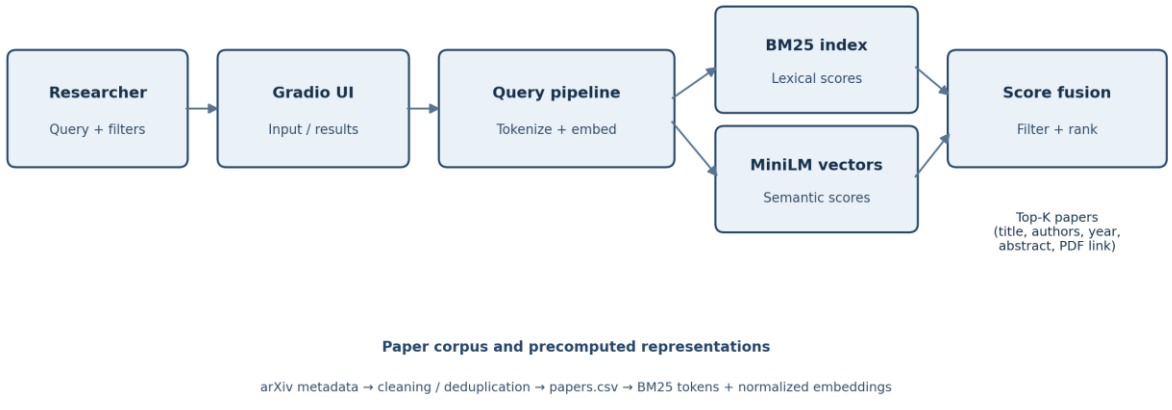


Figure 1. Logical architecture of the hybrid paper retrieval engine.

3.4 Data model

Field	Purpose	Important interpretation
paperId	Identifier obtained from arXiv entry URL	Useful for deduplication; version suffixes may distinguish revisions.
title	Paper title	Displayed as the primary result label.
abstract	Abstract text	Used with the title for both retrieval branches.
authors	Up to three author names; “et al.” if needed	Presentation field; full author list is not preserved in the cleaned string.
year	Publication year, coerced to integer	Missing values become 0 and will usually be excluded by normal year filters.
url	PDF URL from arXiv metadata	Should be validated before rendering as a clickable link.
venue	Set to “ArXiv” for fetched items	Not a peer-reviewed venue claim.
citationCount	Set to 0 by this data collector	Placeholder only; it is not a measured citation count.

3.5 Data integrity and field semantics

The corpus is a snapshot of search results, not a complete registry of research on the selected topics. The paper identifier, title, abstract, authors, year, and PDF link are the fields most directly supported by the returned arXiv records. In contrast, the venue label is assigned by the script and the citationCount value is a placeholder. These fields must be described accurately if they are exposed in the interface or used in later analysis. A valid paper year is important for filtering, and a non-empty abstract is required for the current retrieval representation.

3.6 Core design decisions

The design precomputes document embeddings because paper text remains fixed between queries, avoiding repeated encoding cost. It retains the full corpus in memory so the prototype can be run from a notebook without a separate database or search service. A CSV snapshot keeps the collection inspectable. These trade-offs minimize setup complexity but limit scalability, reproducibility across live API refreshes, and support for concurrent users. In a production architecture, collection, indexing, retrieval, and presentation would be separated into explicit modules with versioned data and observable error handling.

4. Methodology and Ranking Design

4.1 Data acquisition

The collection stage iterates through the topic list and issues one arXiv search per topic using a query of the form all:"topic phrase". Each request asks for a maximum of 50 records and sorts by relevance. The result record is converted to a dictionary containing paper identifier, title, cleaned abstract, author objects, publication year, PDF URL, venue label, and a citation-count placeholder. Errors for an individual topic are printed, and the collection loop continues. A two-second pause is inserted between topics.

Acquisition caveat

The request limit is a maximum per topic, not a guarantee of 500 unique papers. Search overlap, network failures, server-side limits, and later deduplication can reduce the corpus size. The arXiv API documentation encourages clients to be considerate with repeated requests; increasing the inter-request delay and adding retry/backoff handling would improve robustness [5].

4.2 Cleaning and deduplication

The author list is transformed into a readable string showing at most three names. The year and citationCount columns are converted to numeric values, missing citation counts become zero, and records with missing paperId, title, or abstract are removed. Empty abstracts are excluded. Duplicate paper identifiers and duplicate exact titles are removed in sequence, after which the dataframe index is reset and papers.csv is saved. The venue field is populated with “ArXiv” for fetched records.

This cleaning step is intentionally lightweight. It does not normalize Unicode, canonicalize punctuation, merge title variants, or collapse different arXiv versions to one conceptual paper. As a result, near-duplicate items can remain. Also, the fallback dataset contains one illustrative record and assigns a citationCount of 90000; this should be regarded as hard-coded demonstration metadata, not a verified current metric. The main collector itself does not obtain citation counts.

4.3 Text representation

For every paper, the retrieval text is title + one space + abstract. The tokenize function converts text to lowercase and extracts sequences matching the regular expression \b\w+\b. This simple tokenizer is deterministic and dependency-light, but it may split punctuation-rich technical identifiers in ways that are not ideal for research search. No field weighting is applied; title terms and abstract terms enter the same combined text. A future version could weight title terms more heavily or index title and abstract as separate fields.

4.4 BM25 branch

The tokenized paper strings are passed to BM25Okapi once during initialization. At query time, query tokens are extracted using the same tokenizer, and bm25.get_scores returns one score for each corpus record. The function normalize_scores then applies min–max normalization:

$$s_norm = (s - \min(s)) / (\max(s) - \min(s))$$

If all scores are identical, the function returns a zero array to avoid division by zero. Thus, normalized BM25 values are relative to the current corpus score distribution, not probabilities or calibrated relevance estimates.

4. Methodology and Ranking Design

4.5 Semantic branch

The script loads sentence-transformers/all-MiniLM-L6-v2 and computes an embedding for each title-plus-abstract string. The embeddings are normalized once, allowing the query vector to be normalized and compared using a dot product. The code wraps the query in a one-element list before encoding, yielding a batch dimension and avoiding shape inconsistencies. The embedding array is retained in memory for query-time use, so document vectors do not need to be recomputed for every search.

Model-length consideration

The model card states that input longer than 256 word pieces is truncated by default [3]. Since the project encodes title plus abstract as one sequence, the latter part of long abstracts may not contribute to the stored semantic vector. This should be measured or mitigated with truncation-aware preprocessing, separate title/abstract embeddings, or chunked text representations.

4.6 Score fusion

For each query, the semantic dot-product scores and BM25 scores are independently normalized over the full indexed corpus. The final hybrid score is calculated as:

$$H(d, q) = \alpha \cdot S_{sem}(d, q) + (1 - \alpha) \cdot S_{BM25}(d, q)$$

Here, α lies between 0 and 1. A value of 0.7 gives the semantic score a 70% weight and the lexical score a 30% weight. The score is a ranking heuristic; it is neither a probability nor a direct measure of scientific quality. Because min–max normalization is performed over the entire corpus before year filtering, the displayed component scores may be influenced by papers outside the selected year range even though those papers are not returned.

4.7 Year filter and Top-K

The selected minimum and maximum years are converted to integers, and the cleaned dataframe is filtered inclusively: min_year ≤ year ≤ max_year. The function then walks through the retained dataframe rows, uses the original dataframe index to look up the precomputed score arrays, sorts candidates in descending hybrid-score order, and returns the requested Top-K records. Since the dataframe is reset after cleaning and not subsequently reindexed, row indices remain aligned with the indexes built earlier in the script. The UI currently limits Top-K to five or ten.

4.8 Runtime and complexity considerations

Let N be the number of papers, L the average token count, and D the embedding dimension. Corpus preparation tokenizes all documents and constructs the BM25 statistics and N dense vectors once. At query time, BM25 scoring depends on the query terms and index representation; dense scoring computes a similarity between the query vector and each document vector, requiring approximately

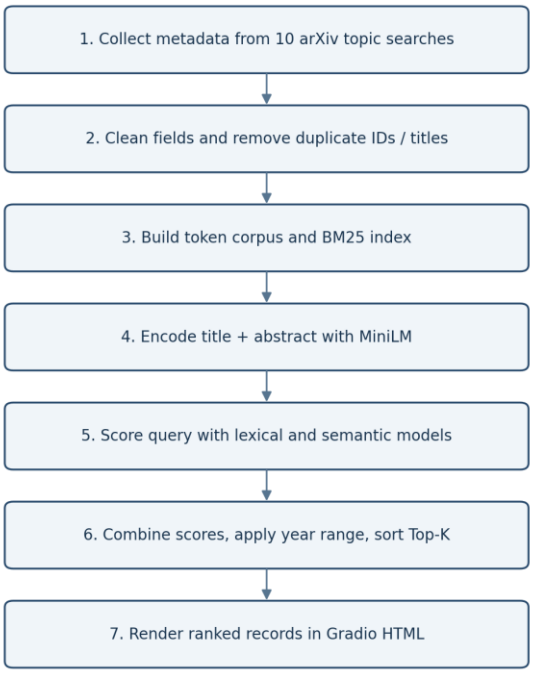
O(ND) arithmetic for a full scan. The current implementation keeps all embeddings in memory and linearly scores the corpus, which is appropriate for a small demonstration corpus. A larger corpus would benefit from approximate nearest-neighbour search, vector storage, caching, or a dedicated search engine.

5. Implementation Details

5.1 Technology stack

Component	Library / service	Role in the project
Data source	arXiv API via arxiv Python package	Search and retrieve paper metadata.
Data preparation	pandas, NumPy, regular expressions	Tabular cleaning, numeric conversion, tokenization, and arrays.
Lexical retrieval	rank_bm25 / BM25Okapi	Token-based score computation.
Semantic retrieval	sentence-transformers, all-MiniLM-L6-v2	Dense embeddings for query and paper text.
Similarity utilities	sentence_transformers.util	Embedding normalization and dot-product scoring.
Evaluation / plots	NumPy, matplotlib; sklearn imported	MRR calculation and α-versus-MRR plot; sklearn ndcg_score is imported but not used.
Web interface	Gradio	Search controls and HTML result presentation.
Persistence	CSV file: papers.csv	Snapshot of the cleaned metadata used for the run.

5.2 End-to-end workflow



Search routine

The hybrid_search function first returns an empty list for a blank query or an empty year-filtered dataframe. It computes full-corpus BM25 scores, normalizes them, computes one semantic query embedding, calculates dot products against paper vectors, and normalizes those scores. For each paper surviving the year filter, it builds a result dictionary with the paperId, title, component scores, hybrid score, and a copy of the row metadata. The result list is sorted descending by hybrid score and sliced to Top-K.

5.4 Gradio interface

The user interface is created with gr.Interface. Its controls are a research-query textbox, minimum-year slider, maximum-year slider, Top-K radio selector, and semantic-weight slider. A result-rendering function produces HTML cards showing rank, title, authors, year, venue, hybrid/semantic/BM25 scores, an abstract excerpt, and a PDF link. If no papers are found in the selected range, it returns a no-results message. The application launches with share=True, which is convenient in a hosted notebook but creates a publicly reachable share tunnel while active [6].

5.5 Installation and execution

The submitted notebook begins with pip installation of sentence-transformers, rank_bm25, gradio, pandas, NumPy, requests, scikit-learn, matplotlib, and arxiv. In a clean environment, it is preferable to use a virtual environment and record exact package versions after a successful run. The data collection and embedding generation are performed at startup; consequently, initial launch time depends on network availability, the number of unique records, and the hardware available for encoding.

Reproducibility note

The report does not state a fixed corpus count, execution duration, or metric value because none was supplied as runtime output. For an auditable run, preserve papers.csv, the query list, dependency versions, model identifier, timestamp, α sweep values, and the generated plot together.

5.6 Data persistence and operational behavior

The cleaned metadata is persisted to papers.csv, but the BM25 index and embedding matrix are built in memory each time the notebook runs. Consequently, a new launch typically repeats both model loading and index construction. The script does not currently save model vectors or an index artifact, and it does not provide a scheduled refresh workflow. Separating acquisition from search startup would reduce launch time and make experiments easier to reproduce against a fixed corpus snapshot.

6. Evaluation Methodology and Test Plan

6.1 Current evaluation procedure

The code defines three fixed evaluation queries: “deep neural networks for vision,” “attention mechanism,” and “malware detection.” For each query, it ranks the corpus with BM25 and marks the top three papers with positive BM25 scores as the ground-truth set. It then runs the hybrid search for α values 0.3, 0.5, 0.7, and 0.8, with a year range from 1900 through 2026 and Top-K = 5. For each retrieved ranking, it searches for the first paper whose identifier is in the pseudo-ground-truth set; if one is found at rank r, the reciprocal rank is 1/r, otherwise it is zero. MRR is the mean reciprocal rank across queries with non-empty pseudo-ground truth.

6.2 Metric definition

$$\mathbf{RR(q) = 1 / \text{rank of the first relevant result}}$$

$$\mathbf{MRR = (1 / |Q|) \times \Sigma\ RR(q)}$$

MRR emphasizes whether at least one judged-relevant paper appears near the top of a result list. It does not measure the quality of every returned result. Normalized Discounted Cumulative Gain (NDCG) would support graded relevance judgments, but scikit-learn’s ndcg_score is imported in the notebook and is not called by the submitted evaluation code [7].

6.3 Evaluation validity

The current procedure should be treated as a pipeline sanity check, not as an unbiased estimate of retrieval quality. Its reference set is generated directly by BM25, and the reference documents are defined as BM25’s top results. This creates an evaluation preference for lexical retrieval: a hybrid system receives credit for returning the documents selected by the same lexical method used to form the pseudo-labels. The three queries are also manually chosen and do not constitute a representative benchmark. No human relevance labels, judged query-document pairs, held-out dataset, or confidence intervals are provided.

Interpretation rule

Do not report a high MRR from this setup as proof that the system retrieves scientifically relevant papers. It indicates agreement with the current BM25-derived pseudo-labels. A valid comparative evaluation needs independently judged relevance labels or a published benchmark with explicit query-document relevance assessments.

6. Evaluation Methodology and Test Plan (continued)

6.5 Functional test cases

Test ID	Input / condition	Expected behavior to verify
TC-01	Blank or whitespace-only query	Return no results without a scoring exception.
TC-02	Query terms occurring in a paper title	Lexical branch should assign positive BM25 score when tokens match.
TC-03	Paraphrased concept with few exact matches	Semantic branch should rank conceptually related text; inspect results manually.
TC-04	Minimum year greater than maximum year	Currently likely returns no papers; add explicit validation and a user-facing message.
TC-05	Year exactly equal to min_year or max_year	Confirm inclusive filtering behavior.
TC-06	Alpha = 0	Hybrid score should equal normalized BM25 score.
TC-07	Alpha = 1	Hybrid score should equal normalized semantic score.
TC-08	No papers within selected years	Display the no-results message.
TC-09	API failure for one topic	Continue other topics and log the failed topic.
TC-10	Duplicate paper identifier or exact title	Keep only one record after cleaning.
TC-11	Fallback path with one record	Application should start; score arrays may normalize to all zeros if every score ties.
TC-12	Title / abstract contains HTML-like text	Confirm output is safely escaped and cannot inject markup or script.

6.6 Proposed evaluation dataset

Create a query set containing different information needs, such as broad topic discovery, technical-method queries, synonym/paraphrase queries, acronym queries, and narrow application queries. At least two reviewers should label a pool of candidate papers for each query using a defined relevance scale—for example, 0 = not relevant, 1 = partially relevant, 2 = relevant, and 3 = highly relevant. Record disagreements and use adjudication or report inter-annotator agreement. Split development queries from test queries so that α is tuned on development data and the final metric is calculated once on the held-out set.

6.7 Metrics to report

Report MRR@5 for first-relevant-result behavior, NDCG@5 or NDCG@10 for graded relevance and ranked-position sensitivity, Recall@K for coverage of known relevant papers, and Precision@K for the fraction of top results judged relevant. Also report query latency, indexing time, corpus size, failure rate during data collection, and memory consumption. For each metric, compare BM25-only, semantic-only, and hybrid runs using the same test queries and the same relevance judgments. Show the mean and, where feasible, variation across queries rather than only a single aggregate number.

7. Results, Discussion, and Limitations

7.1 What the supplied code establishes

From static inspection of the source code, the prototype implements all major stages of an end-to-end retrieval demo: multiple arXiv searches, metadata shaping, duplicate removal, CSV export, BM25 indexing, dense embedding generation, a weighted score, inclusive year filtering, Top-K sorting, an α sweep, an MRR plot, and a Gradio-based user interface. These are implementation capabilities, not empirical evidence that the returned papers are relevant or that the hybrid method outperforms a baseline.

7.2 Numeric results status

Result item	Status in supplied material	Required action
Final unique corpus size	Not shown; depends on live API responses and deduplication.	Record len(df) after cleaning.
MRR for each α	The code computes the series but no values or console output were supplied.	Print and save a table of α and MRR.
Best-performing α	Cannot be determined from code alone.	Select only after evaluation on independent relevance labels.
Search latency	Not measured.	Time query encoding, scoring, filtering, and rendering separately.
NDCG	Not computed.	Use independent graded labels and call ndcg_score correctly.
User-study feedback	Not included.	Conduct task-based testing with target users.

7.3 Strengths

- Complementary retrieval signals are combined in a simple, interpretable score equation.
- Document embeddings are precomputed, avoiding repeated corpus encoding on every query.
- Publication-year constraints make it possible to narrow search results temporally.
- The interface exposes α and Top-K, supporting exploratory comparisons without code changes.
- Metadata is persisted in CSV, making the collected corpus inspectable and reusable.

7.4 Limitations and risks

- Pseudo-ground-truth evaluation is biased toward BM25 and cannot establish independent relevance quality.
- The fallback dataset is only one paper, so the interface may appear to work while providing almost no useful coverage.
- citationCount is a placeholder (zero for fetched records); it must not be interpreted as real citation data.
- The imported ndcg_score function is not used, and numeric metric values are neither printed nor saved by the shown code.
- The model truncates long input sequences; the title-plus-abstract text may be partly ignored by the semantic branch [3].

- The UI inserts external metadata into HTML strings without explicit escaping; retrieved content should be sanitized.
- The public share option has no authentication configured in this code and should be used with care [6].
- A two-second pause, basic exception handling, and no retry/backoff may be insufficient for robust repeated collection [5].
- Global score normalization and zero-variance cases can make score magnitudes hard to interpret across different corpora.
- The dataset is a topic-oriented arXiv sample, not a complete or unbiased corpus of scholarly literature.

8. Recommendations and Future Enhancements

8.1 Improve data collection reliability

Use a persistent collection cache so that routine launches do not need to repeat every API request. Add bounded retries with exponential backoff for transient errors, log the query and timestamp for each batch, and respect current arXiv API guidance. Store raw metadata separately from cleaned data so that cleaning logic can be changed without repeating network calls. Consider normalizing arXiv IDs and grouping paper revisions to avoid duplicated versions of the same work. Validate that the fetched URL points to an allowed arXiv host before presenting it in the UI.

8.2 Improve retrieval quality

Separate title and abstract representations and test field-aware weighting. Add stop-word handling or stemming only after checking whether it helps technical terminology; preprocessing choices should be evaluated rather than assumed. Consider a reciprocal-rank-fusion variant as an alternative to combining min–max normalized scores, because score scales can vary by query. For larger corpora, use a vector index such as an approximate nearest-neighbour library and combine its candidate set with BM25 candidates. A cross-encoder reranker may improve precision at the top of the list, at the cost of additional latency.

8.3 Make evaluation scientifically defensible

Create an explicit query-document relevance dataset, preserve the labels independently of both ranking systems, and evaluate on held-out queries. Report BM25-only, semantic-only, and hybrid results side by side. Use at least MRR@K and NDCG@K; include Recall@K or Precision@K if appropriate for the task. Run a controlled α grid and choose the weight on development data. Save the complete per-query metrics so improvements can be inspected rather than inferred from one averaged plot.

8.4 Improve product behavior and safety

Escape title, author, venue, and abstract text with an HTML-escaping utility before inserting it into result markup. Allow only validated HTTPS URLs from approved domains in anchor tags. Validate the year range, provide clear errors for invalid input, and handle missing/empty metadata explicitly. Add result pagination, snippets that highlight matched terms, sorting by year, and a concise

explanation of why a paper was retrieved. If the application is shared beyond a private notebook, configure authentication, rate limits, and appropriate access controls.

8.5 Suggested next development stages

Stage	Deliverable	Completion criterion
1. Reliability	Cached, retry-aware collector and dataset audit log	Repeatable corpus snapshot with counts and failure logs.
2. Evaluation	Independently judged query-paper benchmark	Reproducible metrics for BM25, semantic, and hybrid runs.
3. Retrieval tuning	Field-aware weighting and alternative fusion	Improvement on held-out ranking metrics without query leakage.
4. Scalability	Approximate vector index and candidate reranking	Measured latency and memory profile on a larger corpus.
5. Deployment	Sanitized output, validation, authentication, monitoring	Safe deployment checklist passes; access behavior is documented.

9. Conclusion

This project presents a compact hybrid research-paper retrieval engine built around metadata collected from arXiv. Its central design combines a conventional BM25 index with dense semantic representations from all-MiniLM-L6-v2. A weighted sum allows the semantic contribution to be adjusted, while year filtering and Top-K selection make search behavior accessible through a Gradio interface. The pipeline is straightforward to inspect and provides a practical foundation for learning and experimenting with information retrieval.

The most important conclusion is methodological: a working retrieval pipeline is not the same as a validated retrieval system. The MRR calculation in the current code uses BM25's own top-ranked papers as pseudo-ground truth. The resulting plot can help inspect how the hybrid ranking agrees with that lexical reference, but it cannot demonstrate independent relevance quality or establish a generally optimal α. The report therefore avoids asserting unobserved values or ranking improvements.

The next priority should be to construct an independently judged evaluation set, measure BM25-only and semantic-only baselines, and test the hybrid ranker on held-out queries. Improvements to metadata provenance, model truncation handling, HTML sanitization, API retry behavior, and public-interface access control would also strengthen reliability. With these changes, the prototype can evolve from a useful classroom demonstration into a more defensible research-support tool.

Final takeaway

The project demonstrates the integration of open scholarly metadata, lexical retrieval, semantic embeddings, rank fusion, and an interactive interface. Claims about effectiveness should be made only after independent relevance evaluation.

References

[1] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009. doi: 10.1561/1500000019. Available: https://doi.org/10.1561/1500000019

[2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proceedings of EMNLP-IJCNLP, 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410. Available: https://aclanthology.org/D19-1410/

[3] Sentence Transformers, “sentence-transformers/all-MiniLM-L6-v2: Model Card,” Hugging Face. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (accessed 22 October 2026).

[4] D. Brown, “rank_bm25: A Two-Line Search Engine,” GitHub repository and README. Available: https://github.com/dorianbrown/rank_bm25 (accessed 22 October 2026).

[5] arXiv, “arXiv API User Manual,” arXiv documentation. Available: https://github.com/arXiv/arxiv-docs/blob/develop/source/help/api/user-manual.md (accessed 22 October 2026).

[6] Gradio, “Interface,” Gradio Documentation. Available: https://www.gradio.app/docs/gradio/interface (accessed 22 October 2026).

[7] scikit-learn developers, “sklearn.metrics.ndcg_score,” scikit-learn API Documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.ndcg_score.html (accessed 22 October 2026).

Source-code basis

Primary implementation reviewed: the Python notebook/script supplied in the user prompt, including the collection, cleaning, indexing, hybrid_search, evaluate_alpha, plot, and Gradio interface sections. No external runtime logs, result screenshots, papers.csv file, or generated metric values were supplied alongside the code.